TECHNOVA SOLUTIONS PVT. LTD.

ENGINEERING HANDBOOK

Document ID	TN-ENG-HBK-001
Document Type	Engineering Handbook
Policy Owner	Engineering
Approving Authority	VP Engineering / CTO
Classification	Internal — Engineering
Version	1.0
Effective Date	01 April 2026
Review Cycle	Annual
Status	Active
Supersedes	New Handbook

1. Purpose

The Engineering Handbook defines the working practices, responsibilities, technical expectations, development principles, and operational standards for the Engineering organization at TechNova Solutions Pvt. Ltd. (“TechNova” or “the Company”).

It provides a common reference for engineers, technical leads, engineering managers, architects, and other personnel involved in software and technology delivery.

2. Scope

This handbook applies to employees and authorized contributors working within or with the Engineering organization. Specific project, security, architecture, coding, deployment, and testing requirements may be defined in separate Engineering policies and standards.

3. Engineering Mission

Engineering is responsible for designing, building, testing, operating, improving, and maintaining reliable technology products and internal systems that support TechNova's business objectives.

- Deliver useful and maintainable software.
- Build secure and reliable systems.
- Use engineering practices that support quality and sustainable delivery.
- Continuously improve technical capability and operational maturity.
- Collaborate effectively across Engineering and business functions.

4. Engineering Principles

- Customer and business value
- Quality and maintainability
- Security by design
- Reliability and operational ownership

- Automation and repeatability
- Evidence-based technical decisions
- Documentation and knowledge sharing
- Continuous learning and improvement
- Respectful and inclusive collaboration

5. Engineering Organization

Engineering may be organized into product teams, platform teams, infrastructure functions, quality functions, architecture groups, security engineering, data or AI teams, and other specialized groups.

The exact organizational structure may change as TechNova's products and technology requirements evolve.

6. Engineering Roles

Role	Typical Responsibilities
Software Engineer	Design, implement, test, document, troubleshoot, and maintain software.
Senior Software Engineer	Own complex technical work, mentor engineers, and contribute to design and technical decisions.
Technical Lead	Provide technical direction for a team or workstream and coordinate design and delivery.
Engineering Manager	Lead people, planning, delivery, performance, and engineering execution.
Architect	Guide system-level technical design, architecture decisions, and technical standards.
Site / Platform Engineer	Maintain infrastructure, deployment platforms, reliability, automation, and operational capabilities.

7. Engineering Responsibilities

- Understand assigned requirements and technical objectives.
- Produce maintainable and appropriately tested implementations.
- Follow approved coding, Git, review, deployment, and testing practices.
- Protect Company and customer information.
- Raise technical risks early.
- Document important technical decisions and operational knowledge.
- Participate in incident response and corrective actions when required.

8. Development Lifecycle

Engineering work generally follows a lifecycle from requirement understanding through design, implementation, review, testing, deployment, operation, and maintenance.

- Understand the problem and acceptance criteria.
- Assess dependencies and risks.
- Design an appropriate solution.
- Implement changes using approved development practices.
- Review and test changes.
- Deploy through approved mechanisms.
- Monitor behavior and address defects or operational issues.

9. Requirements and Planning

Engineering should clarify requirements before implementation where practical. Ambiguous requirements should be identified and resolved with the relevant product, business, or technical stakeholders.

Engineering estimates and plans should communicate assumptions, dependencies, risks, and uncertainty rather than presenting unsupported precision.

10. Technical Design

Material technical changes should receive an appropriate level of design consideration before implementation.

- Problem statement and objectives
- Architecture or component boundaries
- Interfaces and dependencies
- Data flows and storage
- Security considerations
- Performance and scalability considerations
- Failure modes and recovery
- Operational and monitoring requirements

11. Architecture Principles

- Prefer simple designs that meet current requirements.
- Avoid unnecessary coupling.
- Define clear ownership of services and components.
- Design for appropriate failure handling.
- Consider security, observability, maintainability, and operational cost.
- Document significant architecture decisions.

12. Coding Standards

Engineers must follow the approved Coding Standards for language-specific style, naming, structure, error handling, testing, documentation, and maintainability.

The separate Coding Standards document is authoritative for detailed coding rules.

13. Version Control

All production software changes should be maintained in approved version-control repositories.

- Use meaningful branch and commit practices.
- Keep commits understandable and related to the intended change.
- Do not commit credentials, secrets, or restricted information.
- Use approved repository permissions.
- Preserve appropriate history and review records.

14. Git Workflow

Engineering teams must follow the approved Git Workflow for branching, pull requests, merging, releases, and related repository activities.

The separate Git Workflow document defines the team's detailed workflow.

15. Code Review

Material code changes should receive appropriate peer or technical review before integration, according to project requirements.

- Review correctness and intended behavior.
- Consider security and data-handling implications.
- Check maintainability and clarity.
- Consider testing adequacy.
- Identify performance or reliability concerns where relevant.
- Record or resolve significant review comments.

16. Testing

Engineering is responsible for building appropriate confidence in software before release.

- Unit testing
- Integration testing
- System or end-to-end testing where appropriate
- Regression testing
- Performance testing where required
- Security testing where required

17. Testing Standards

Projects must follow the approved Testing Standards for test organization, quality expectations, environments, automation, and release confidence.

18. Defect Management

Defects should be recorded, prioritized, investigated, and resolved according to severity and business impact.

- Critical production issues receive appropriate urgency.
- Root causes should be investigated for significant recurring defects.
- Fixes should include appropriate regression coverage.
- Known limitations should be communicated when they cannot be immediately resolved.

19. Dependency Management

Engineering teams should maintain awareness of software dependencies and their security, compatibility, licensing, and maintenance status.

Unnecessary or obsolete dependencies should be removed or replaced where practical.

20. Third-Party Components

Third-party libraries, frameworks, APIs, and services should be evaluated for suitability, licensing, security, maintenance, and compatibility before adoption.

Teams must comply with approved software licensing and procurement requirements.

21. Security Engineering

Security must be considered throughout the development lifecycle.

- Use appropriate authentication and authorization.
- Protect secrets and credentials.
- Validate and sanitize untrusted input.
- Protect sensitive information.
- Apply secure dependency and patch practices.
- Address identified vulnerabilities according to risk.

22. Data Handling

Engineering systems must handle data according to its classification and applicable security and privacy requirements.

Sensitive information should not be copied into development or test environments unless appropriately authorized and protected.

23. Artificial Intelligence and Machine Learning

Engineering teams developing or using AI and machine-learning systems must follow approved AI governance, security, privacy, and data-handling requirements.

- Use approved datasets and data sources.
- Protect sensitive information used in training, evaluation, retrieval, or inference.
- Evaluate model outputs for accuracy and relevant risks.
- Document important model, prompt, retrieval, or evaluation assumptions.
- Monitor AI systems appropriately after deployment.

24. Enterprise Knowledge Systems

Engineering systems that support enterprise search, retrieval-augmented generation, document indexing, or AI assistants must enforce authorization before protected content is supplied to downstream processing.

Role, department, document, and other applicable permissions must be respected throughout ingestion, indexing, retrieval, context construction, and response generation.

25. Configuration Management

Application and infrastructure configuration should be managed through approved mechanisms.

- Separate configuration from application code where appropriate.
- Protect secrets.
- Track material configuration changes.
- Use environment-specific configuration safely.
- Avoid undocumented manual production changes.

26. Infrastructure and Cloud

Infrastructure should be provisioned and managed using approved tools and practices.

- Apply least-privilege access.
- Use approved cloud accounts and resources.

- Maintain appropriate backups and recovery capabilities.
- Monitor critical resources.
- Document significant infrastructure dependencies.

27. Deployment

Production deployments must follow the approved Deployment Policy.

- Changes should be traceable to an approved change or delivery process.
- Required testing and review should be completed.
- Deployment risks should be considered.
- Rollback or recovery procedures should exist where appropriate.

28. Release Management

Teams should coordinate releases to reduce operational risk and ensure relevant stakeholders understand material changes.

Release information should identify significant changes, known risks, dependencies, and required operational actions where appropriate.

29. Observability

Production systems should have appropriate logging, metrics, traces, alerts, or other observability mechanisms based on system criticality.

Logs must not unnecessarily expose sensitive information.

30. Reliability and Availability

- Identify important failure modes.
- Use appropriate timeouts, retries, and fallback behavior.
- Monitor critical dependencies.
- Define recovery procedures for important services.
- Learn from significant incidents and outages.

31. Incident Response

Engineering teams must participate in incident response for systems they own or support.

- Acknowledge and assess incidents promptly.
- Communicate status through approved channels.
- Prioritize restoration of service.
- Preserve relevant evidence and logs.
- Document significant incidents and corrective actions.

32. On-Call Responsibilities

Where an on-call model applies, assigned engineers are expected to remain reasonably available during the defined on-call period and follow escalation procedures.

Teams should design on-call practices to support sustainable operations and reduce repeated avoidable incidents.

33. Technical Debt

Technical debt should be identified and managed as part of engineering planning.

Teams should distinguish between acceptable temporary trade-offs and technical debt that creates significant reliability, security, delivery, or maintenance risk.

34. Documentation

Engineering documentation should be maintained for information necessary to operate, maintain, understand, and evolve systems.

- Architecture and major design decisions
- Service ownership
- Operational procedures
- Deployment and rollback procedures
- Important dependencies
- Known limitations
- Troubleshooting information

35. Knowledge Sharing

Engineers are encouraged to share knowledge through documentation, technical discussions, reviews, mentoring, demonstrations, and communities of practice.

36. Ownership

Services and major technical components should have clear ownership.

Owners are responsible for understanding the service's purpose, dependencies, operational risks, security requirements, and maintenance needs.

37. Cross-Functional Collaboration

Engineering works with Product, Operations, Finance, HR, Security, Administration, and other functions as required.

Technical recommendations should clearly communicate trade-offs and impacts to non-technical stakeholders when relevant.

38. Communication

- Communicate technical risks early.
- Avoid unnecessary jargon when communicating with non-technical stakeholders.
- Document decisions that materially affect delivery or operations.
- Use approved collaboration and communication channels.

39. Engineering Meetings

Teams may use planning meetings, stand-ups, design reviews, retrospectives, incident reviews, demos, and other forums appropriate to their workflow.

Meetings should have a clear purpose and should support rather than unnecessarily hinder engineering work.

40. Performance and Career Development

Engineering performance is evaluated according to role expectations, quality of work, outcomes, collaboration, technical capability, ownership, and relevant Company performance processes.

Career progression is governed by the Promotion & Career Development Policy and applicable engineering competency expectations.

41. Learning and Development

- Technical courses and certifications
- Internal knowledge-sharing sessions
- Mentoring
- Architecture and design discussions
- Conference or community participation where approved
- Cross-functional or stretch assignments

42. Engineering Environment

Engineering teams should maintain an environment that encourages respectful discussion, constructive disagreement, experimentation, learning, and accountability.

Technical disagreement should focus on evidence, requirements, constraints, and trade-offs rather than individuals.

43. Engineering Ethics

- Do not knowingly introduce insecure or malicious functionality.
- Do not conceal material technical risks from responsible stakeholders.
- Do not falsify test results, performance measurements, or engineering records.
- Do not misuse production access.
- Do not knowingly expose confidential information.

44. Production Access

Production access must be limited to authorized personnel with a legitimate operational need and managed according to the Access Control Policy.

Engineers must not use production access for unauthorized experimentation or personal purposes.

45. Change Management

Material production changes should follow approved change-management procedures appropriate to system risk.

Emergency changes may use an expedited process but should be documented and reviewed afterward where required.

46. Business Continuity

Engineering teams responsible for critical systems should understand applicable continuity and recovery arrangements and participate in recovery exercises or testing where required.

47. Asset and Equipment Use

Engineering equipment, development environments, credentials, and technical resources are Company assets and must be used responsibly and according to security requirements.

48. Departure and Handover

When an engineer leaves a team or the Company, appropriate handover should include system ownership, documentation, active work, credentials or access transitions, open risks, and operational knowledge.

Access should be modified or revoked according to the Access Control Policy.

49. Engineering Handbook Relationship to Detailed Policies

This handbook provides the overall Engineering framework. Detailed requirements are defined in the Engineering documents shown in the Company's knowledge structure, including Coding Standards, Git Workflow, Code Review Guidelines, Deployment Policy, and Testing Standards.

Where a detailed approved standard provides a more specific requirement, engineers should follow that standard.

50. Frequently Asked Questions

Where should I find coding rules?

Refer to the approved Coding Standards document.

What Git process should I follow?

Follow the approved Git Workflow for branching, commits, pull requests, merging, and releases.

Does every change require code review?

Review requirements depend on the applicable Engineering workflow and project risk, but material changes should receive appropriate review.

Who owns a production service?

Each service or major component should have a clearly identified owner or owning team.

Can I test experimental code in production?

Production systems should not be used for unauthorized experimentation. Follow approved testing and change processes.

What should I do if I discover a serious production issue?

Follow the applicable incident-response and escalation procedures and communicate the issue promptly.

Can I copy production data into development?

Only when explicitly authorized and appropriately protected under applicable data-security requirements.

Can I use an AI tool with Company source code?

Only when the tool and use case are approved and all applicable security, confidentiality, privacy, and AI requirements are satisfied.

How should technical debt be handled?

Identify it, communicate its impact, and prioritize remediation according to risk and business value.

Where are deployment rules defined?

Follow the approved Deployment Policy for detailed deployment requirements.

51. Handbook Administration

Engineering owns this handbook and reviews it at least annually or when material changes occur in Engineering organization, technology practices, security requirements, or delivery processes.

The latest approved version maintained in the official Company repository is the authoritative version.

52. Revision History

Version	Effective Date	Description	Owner	Approval
1.0	01 April 2026	Initial release of Engineering Handbook	Engineering	VP Engineering / CTO

53. Related Documents

- Coding Standards
- Git Workflow
- Code Review Guidelines
- Deployment Policy
- Testing Standards
- Information Security Policy
- Access Control Policy
- Employee Code of Conduct & Ethics Policy
- Performance Management Policy
- Promotion & Career Development Policy

END OF DOCUMENT

TN-ENG-HBK-001 | Version 1.0 | Internal — Engineering | Engineering